

Technical Documentation

Date: July 24, 2026, **Version:** 1.0

Revision History

Version	Date	Author	Description
1.0	24-Jul-2025	Pranesh J S	Initial draft

Table of Contents

Technical Documentation	1
1. Introduction	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Intended Audience.....	1
1.4 Acronyms and Definitions	2
2. System Overview.....	3
2.1 High-Level Architecture.....	3
2.2 System flow - Key Components	4
2.3 User Roles and Access Levels	5
3. Functional Requirements	5
3.1 Module/Feature Name.....	Error! Bookmark not defined.
3.2 Inputs/Outputs.....	7
3.3 Business Rules	8
3.4 Error Handling.....	8
4. Non-Functional Requirements	8
4.1 Performance	8
4.2 Availability	8
5. Data Model / Database Design	9
5.1 Entity-Relationship Diagram	9
5.2 Tables and Fields	9
5.3 Indexes and Keys	9
5.4 Sample Data	10
6. Integration Points	11
6.1 External Systems	11
6.2 APIs Used	12
6.3 Data Flow Diagram	13
7. Test Strategy	13
7.1 Test Scenarios.....	13
7.2 Traceability to Requirements.....	15
8. Deployment & Configuration	16

8.1 Setup Steps.....	16
8.2 Configuration Files and Parameters.....	16
9. Appendix.....	17
Screenshots	17
Code Snippets	17

1. Introduction

1.1 Purpose

The purpose of the AI Service Desk application is to provide a scalable and efficient backend system for managing customer support tickets while leveraging Artificial Intelligence to improve support operations. The application enables users to create, retrieve, update, and delete support tickets, and integrates with Amazon Bedrock to automatically generate ticket summaries and suggest professional agent responses. Built using FastAPI, SQLAlchemy, PostgreSQL, and a layered architecture, the system emphasizes modularity, maintainability, performance, and testability through comprehensive API design, database management, and automated testing.

1.2 Scope

- Ticket creation, retrieval, update, and deletion (CRUD operations).
- AI-generated ticket summaries and suggested responses.
- Secure and efficient RESTful API development using FastAPI.
- PostgreSQL database management with SQLAlchemy.
- Request and response validation using Pydantic.
- Database migration using Alembic.
- Health monitoring endpoints.
- Unit, integration, end-to-end, and load testing.
- A modular architecture that supports future enhancements and integrations.

1.3 Intended Audience

Audience	Purpose
Backend Developers	Understand the application architecture, APIs, business logic, and database interactions.
QA/Test Engineers	Perform unit, integration, end-to-end, and load testing to ensure application quality.
DevOps Engineers	Deploy, configure, monitor, and maintain the application in different environments.
Project Maintainers	Maintain the codebase, fix bugs, and implement future enhancements.

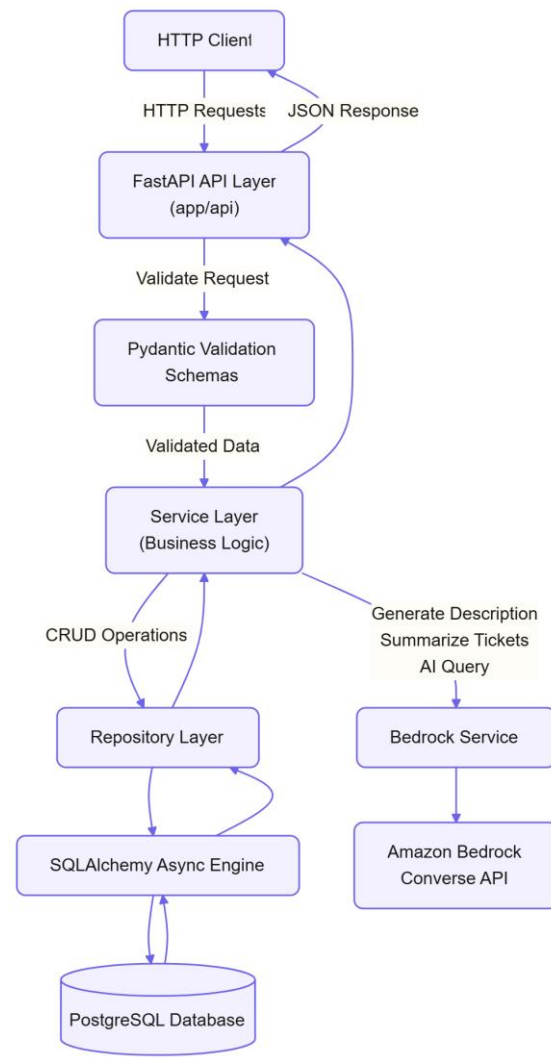
Technical Reviewers	Review the system design, implementation, and overall technical quality of the application.
---------------------	---

1.4 Acronyms and Definitions

- **CRUD** – Create, Read, Update, Delete operations.
- **REST** – Representational State Transfer for web APIs.
- **JSON** – Lightweight data interchange format.
- **ORM** – Object-Relational Mapping.
- **FastAPI** – Python framework for building REST APIs.
- **SQLAlchemy** – Python ORM for database operations.
- **PostgreSQL** – Relational database management system.
- **Pydantic** – Library for data validation and serialization.
- **Alembic** – Database migration tool for SQLAlchemy.
- **Amazon Bedrock** – AWS service for generative AI models.
- **LLM** – Large Language Model.
- **UUID** – Universally Unique Identifier.
- **Pytest** – Python testing framework.
- **Locust** – Load and performance testing tool.
- **ASGI** – Asynchronous Server Gateway Interface.

2. System Overview

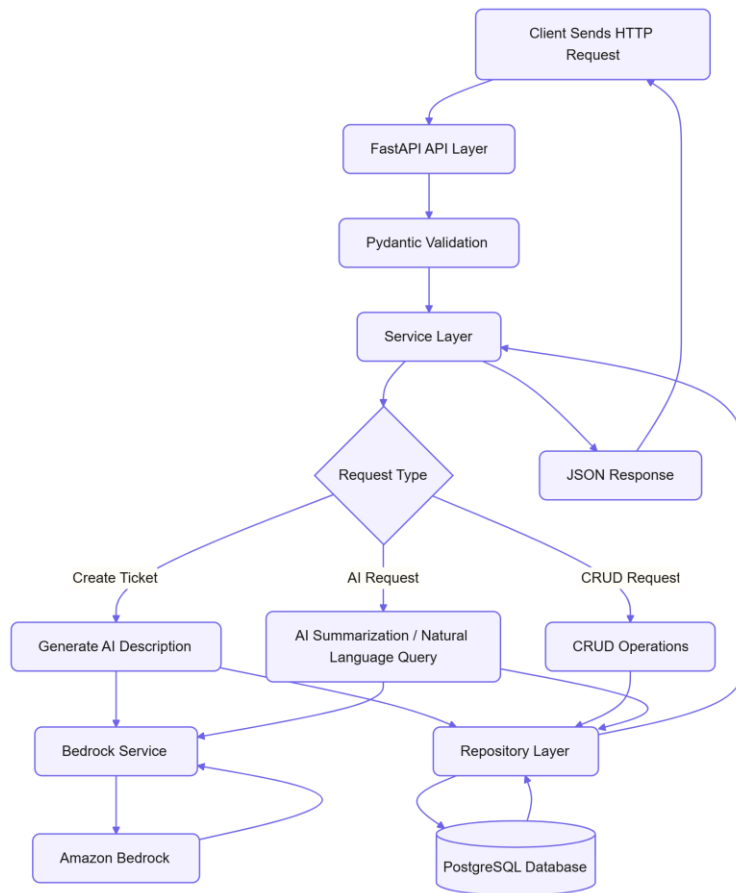
2.1 High-Level Architecture



- **HTTP Client** – Sends requests and receives JSON responses through REST APIs.
- **FastAPI API Layer** – Handles routing, dependency injection, and request/response processing.
- **Pydantic Validation Schemas** – Validates and sanitizes incoming request data before processing.
- **Service Layer** – Implements business rules, coordinates database operations, and communicates with AI services.
- **Repository Layer** – Encapsulates all database CRUD operations using SQLAlchemy Async.
- **SQLAlchemy Async Engine** – Executes asynchronous database queries against PostgreSQL.
- **PostgreSQL Database** – Stores support ticket information and application data.

- **Bedrock Service** – Manages communication with Amazon Bedrock for AI-powered ticket description generation, summarization, and intelligent querying.
- **Amazon Bedrock** – Generates AI responses using foundation models through the Converse API.

2.2 System flow - Key Components



Key Components

- **Client / Frontend** – Sends requests and receives responses.
- **FastAPI** – Routes incoming requests.
- **Controller** – Validates requests and forwards them.
- **Service Layer** – Executes business logic and AI operations.
- **Amazon Bedrock** – Generates AI summaries and suggested responses.
- **Repository** – Handles database operations.
- **PostgreSQL** – Stores application data.

2.3 User Roles and Access Levels

User Role	Permissions
Client/User	Create, retrieve, update, and delete support tickets. Generate AI-powered ticket descriptions during ticket creation. Request AI-based ticket summaries and natural language queries. Access health and readiness endpoints.

3. Functional Requirements

1. Ticket Management Module

Purpose

The Ticket Management module is the core component of the AI-Powered Service Desk. It enables users to create, retrieve, update, and delete support tickets through RESTful APIs while enforcing business rules. It also coordinates AI-powered description generation during ticket creation and serves as the primary interface for ticket management.

Functionality

- Allows users to create new support tickets by providing the title, priority, and status.
- Automatically generates a detailed ticket description using Amazon Bedrock during ticket creation.
- Retrieves individual tickets or a list of tickets with optional filtering based on priority and status.
- Updates existing ticket information while preventing modifications to closed tickets.
- Prevents the creation of duplicate tickets based on business rules.
- Deletes tickets from the PostgreSQL database when requested.
- Stores and retrieves ticket information through the Repository Layer using SQLAlchemy Async.

2. AI Description Generation Module

Purpose

The AI Description Generation module automatically generates meaningful and context-aware ticket descriptions using Amazon Bedrock during ticket creation, reducing manual effort and improving ticket quality.

Functionality

- Accepts the ticket title, priority, and status as input.
 - Uses prompt templates to construct requests for Amazon Bedrock.
 - Invokes the Bedrock Converse API to generate ticket descriptions.
 - Parses and validates AI responses before storing them in the database.
 - Provides fallback mechanisms for handling unexpected AI responses.
-

3. AI Summarization & Query Module

Purpose

The AI Summarization & Query module enables users to summarize support tickets and ask natural language questions about existing tickets using Amazon Bedrock.

Functionality

- Accepts natural language requests from users.
 - Retrieves relevant ticket information from the database when required.
 - Sends ticket context and user queries to Amazon Bedrock.
 - Generates concise ticket summaries and suggested responses.
 - Answers context-aware questions about tickets using AI-generated responses.
-

4. Validation Module

Purpose

The Validation module ensures that all incoming requests satisfy application constraints before any business logic or database operations are executed.

Functionality

- Validates request payloads using Pydantic schemas.
 - Removes unnecessary whitespace from ticket titles.
 - Rejects empty or invalid ticket titles.
 - Ensures request data conforms to the expected schema before processing.
-

5. Performance Monitoring Module

Purpose

The Performance Monitoring module measures the execution time of every HTTP request to help monitor API performance.

Functionality

- Tracks request execution time using custom middleware.
- Adds the **X-Response-Time** header to every HTTP response.
- Assists developers in identifying performance bottlenecks.
- Supports profiling and benchmarking of API endpoints.

6. Health Monitoring Module

Purpose

The Health Monitoring module verifies the availability of the application and its dependent services.

Functionality

- Provides a **/health** endpoint to verify database connectivity.
- Provides a **/ready** endpoint for readiness checks.
- Helps deployment platforms determine application availability.
- Supports production monitoring and health verification.

3.2 Inputs/Outputs

Module	Input	Processing	Output
Ticket Management	Ticket title, priority, and status	Validates input, generates AI description, stores or retrieves ticket data	Ticket details with AI-generated description
AI Description Generation	Title, priority, and status	Sends prompt to Amazon Bedrock and generates a detailed description	AI-generated ticket description
AI Summarization & Query	Natural language query or ticket description	Retrieves relevant ticket data, invokes Amazon Bedrock, and generates a response	Ticket summary or AI-generated answer
Validation	HTTP request payload	Validates schema, trims whitespace, and checks business constraints	Validated request or validation error

Module	Input	Processing	Output
Health Monitoring	Health or readiness request	Checks application and database availability	Service status response
Performance Monitoring	Incoming HTTP request	Measures execution time using middleware	HTTP response with X-Response-Time header

3.3 Business Rules

- Ticket **title** and **description** are mandatory fields.
- Input data is validated using **Pydantic** before processing.
- Only existing tickets can be retrieved, updated, or deleted.
- AI summary and suggested response generation require valid ticket details.
- Database operations are performed only through the **Repository Layer**.
- Business logic is handled exclusively by the **Service Layer**, ensuring separation of concerns.
- Appropriate HTTP status codes and error messages are returned for invalid requests or missing resources.

3.4 Error Handling

- Input validation is performed using **Pydantic**.
- Invalid requests return appropriate **HTTP error status codes**.
- Non-existent tickets return a **resource not found** error.
- Exceptions are handled to prevent application crashes.
- Error messages are returned to the client in JSON format.
- Application logs help in debugging and monitoring errors during development.

4. Non-Functional Requirements

4.1 Performance

- The application is built using **FastAPI** with asynchronous request handling to support high concurrency.
- Database operations are performed using **SQLAlchemy 2.0 Async** with the **asyncpg** PostgreSQL driver for efficient database access.
- A custom middleware measures the execution time of every HTTP request and appends the **X-Response-Time** header to the response.

- Performance profiling is supported through **profile_testing.py**, while load testing is performed using **Locust** to evaluate system behavior under concurrent user loads.

4.2 Availability

- Health endpoint to verify application availability.
- Readiness endpoint to check service dependencies.
- Continuous availability during normal operation.
- No failover or redundancy mechanisms in the current implementation.
- Supports future deployment with load balancing and redundant instances if required.

5. Data Model / Database Design

5.1 Entity-Relationship Diagram

TICKET		
UUID	id	PK
STRING	title	UK
ENUM	priority	
ENUM	status	
TEXT	description	
TIMESTAMP	created_at	
TIMESTAMP	updated_at	

5.2 Tables and Fields

Field Name	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier assigned to each support ticket.
title	String	Unique, Not Null	Brief title describing the support issue. Duplicate titles are not allowed.
priority	Enum	Not Null	Indicates the priority level of the ticket (Low, Medium, High).

Field Name	Data Type	Constraints	Description
status	Enum	Not Null	Represents the current state of the ticket (Open, In Progress, Closed).
description	Text	Not Null	AI-generated detailed description of the support ticket.
created_at	Timestamp	Not Null	Stores the date and time when the ticket was created.
updated_at	Timestamp	Nullable	Stores the date and time of the most recent ticket update.

5.3 Indexes and Keys

Keys

- **Primary Key:** id (UUID) – Uniquely identifies each ticket.
- **Foreign Keys:** None in the current implementation.

Indexing Strategy

- The **Primary Key (id)** is indexed automatically for fast retrieval of tickets.

5.4 Sample Data

ID	Title	Priority	Status	AI-Generated Description	Created At	Updated At
3d6f2c6d-5a2d-4d13-a7a1-8c0f54d5c101	Login Issue	High	Open	User is unable to log in to the application due to repeated authentication failures. Immediate investigation is required to restore access.	2026-07-24 09:15:22	NULL
a8c1d3b2-74d5-48b0-b2a7-f8d1c9e2f345	VPN Connection Error	Medium	In Progress	User experiences intermittent VPN connection timeouts while accessing corporate resources	2026-07-24 10:02:47	2026-07-24 11:30:18

ID	Title	Priority	Status	AI-Generated Description	Created At	Updated At
				remotely. Investigation is currently underway.		
d0e9f5b1-2f74-49c7-9f84-3e6d0a7b8c90	Email Delivery Failure	Low	Closed	Emails sent from the internal mail server were not reaching external recipients due to an SMTP configuration issue. The problem has been resolved and the ticket has been closed.	2026-07-24 08:41:53	2026-07-24 12:18:09

6. Integration Points

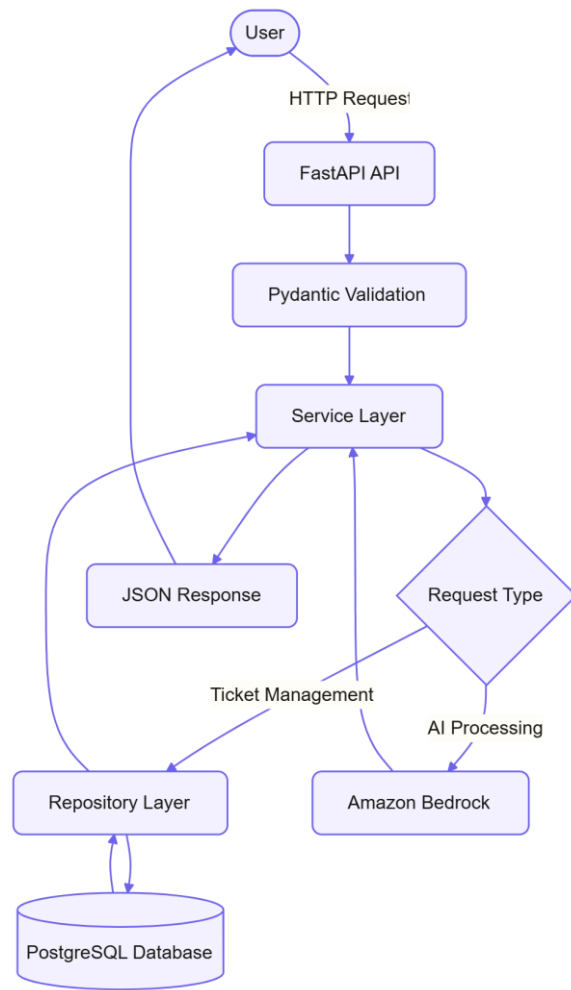
6.1 External Systems

External System	Purpose
Amazon Bedrock	Generates AI-powered ticket descriptions during ticket creation and performs intelligent ticket summarization and natural language querying using foundation models through the Converse API.
PostgreSQL Database	Stores and manages ticket information for CRUD operations.

6.2 APIs Used

HTTP Method	Endpoint	Purpose
GET	/health	Checks the health of the application and verifies active database connectivity.
GET	/ready	Performs a readiness check to determine whether the application is ready to serve requests.
POST	/tickets/ticket	Creates a new support ticket and automatically generates an AI-powered description using Amazon Bedrock.
GET	/tickets/ticket	Retrieves all support tickets with optional filtering based on priority and status.
GET	/tickets/ticket/{id}	Retrieves a specific support ticket using its unique UUID.
PUT	/tickets/ticket/{id}	Updates an existing support ticket while enforcing business rules such as preventing updates to closed tickets.
DELETE	/tickets/ticket/{id}	Deletes a support ticket from the database.
POST	/ai/summarize	Generates AI-powered ticket summaries, suggested responses, or answers natural language questions about existing tickets using Amazon Bedrock.

6.3 Data Flow Diagram



7. Test Strategy

7.1 Test Scenarios & Expected Results

Test ID	Test Scenario	Expected Result	Status
TS-01	Create a new support ticket with valid data	Ticket is created successfully with an AI-generated description.	Pass
TS-02	Retrieve all support tickets	List of all tickets is returned successfully.	Pass
TS-03	Retrieve a ticket using a valid UUID	Requested ticket details are returned.	Pass

Test ID	Test Scenario	Expected Result	Status
TS-04	Update an existing ticket	Ticket information is updated successfully and updated_at is modified.	Pass
TS-05	Delete an existing ticket	Ticket is removed successfully from the database.	Pass
TS-06	Create a ticket with a duplicate title	Request is rejected with a DuplicateTicketError .	Pass
TS-07	Update a ticket whose status is Closed	Request is rejected with a ClosedTicketError .	Pass
TS-08	Create a ticket with an empty or whitespace-only title	Validation fails and an appropriate error response is returned.	Pass
TS-09	Generate AI-powered ticket description	Amazon Bedrock generates and returns a ticket description, which is stored in the database.	Pass
TS-10	Summarize ticket information using the AI endpoint	AI-generated ticket summary and suggested response are returned successfully.	Pass
TS-11	Submit a natural language query through the AI endpoint	The system retrieves relevant ticket information and returns an AI-generated answer.	Pass
TS-12	Access the /health endpoint	Service health and active database connectivity are verified successfully.	Pass
TS-13	Access the /ready endpoint	Application readiness status is returned successfully.	Pass
TS-14	Verify the response-time middleware	Every HTTP response includes the X-Response-Time header.	Pass
TS-15	Execute load testing using Locust	Application handles concurrent requests within acceptable performance limits.	Pass
TS-16	Execute profiling using profile_testing.py	Performance statistics and profiling reports are generated successfully.	Pass

7.2 Traceability to Requirements

Requirement ID	Requirement Description	Requirement Type	Mapped Test Scenario(s)
FR-01	Create a new support ticket with AI-generated description.	Functional	TS-01
FR-02	Retrieve all support tickets.	Functional	TS-02
FR-03	Retrieve a support ticket by UUID.	Functional	TS-03
FR-04	Update an existing support ticket.	Functional	TS-04
FR-05	Delete a support ticket.	Functional	TS-05
FR-06	Prevent duplicate ticket creation.	Functional	TS-06
FR-07	Prevent updates to closed tickets.	Functional	TS-07
FR-08	Validate ticket input data before processing.	Functional	TS-08
FR-09	Automatically generate ticket descriptions using Amazon Bedrock.	Functional	TS-09
FR-10	Generate AI-powered ticket summaries.	Functional	TS-10
FR-11	Answer natural language queries about existing tickets.	Functional	TS-11
FR-12	Verify application health through the /health endpoint.	Functional	TS-12
FR-13	Verify application readiness through the /ready endpoint.	Functional	TS-13
NFR-01	Measure and return API response time using custom middleware.	Non-Functional	TS-14
NFR-02	Support load testing using Locust.	Non-Functional	TS-15

Requirement ID	Requirement Description	Requirement Type	Mapped Test Scenario(s)
NFR-03	Support application profiling using profile_testing.py.	Non-Functional	TS-16

8. Deployment & Configuration

8.1 Setup Steps

The following steps are required to set up the AI Service Desk in the **Development (Dev)** environment.

1. Clone the project repository.
2. Create and activate a Python virtual environment.
3. Install the required dependencies using requirements.txt.
4. Configure environment variables (PostgreSQL connection, AWS credentials, and Amazon Bedrock settings).
5. Run Alembic migrations to create the database schema.
6. Start the FastAPI application using Uvicorn.
7. Verify the application by accessing the **Health** and **Readiness** endpoints.

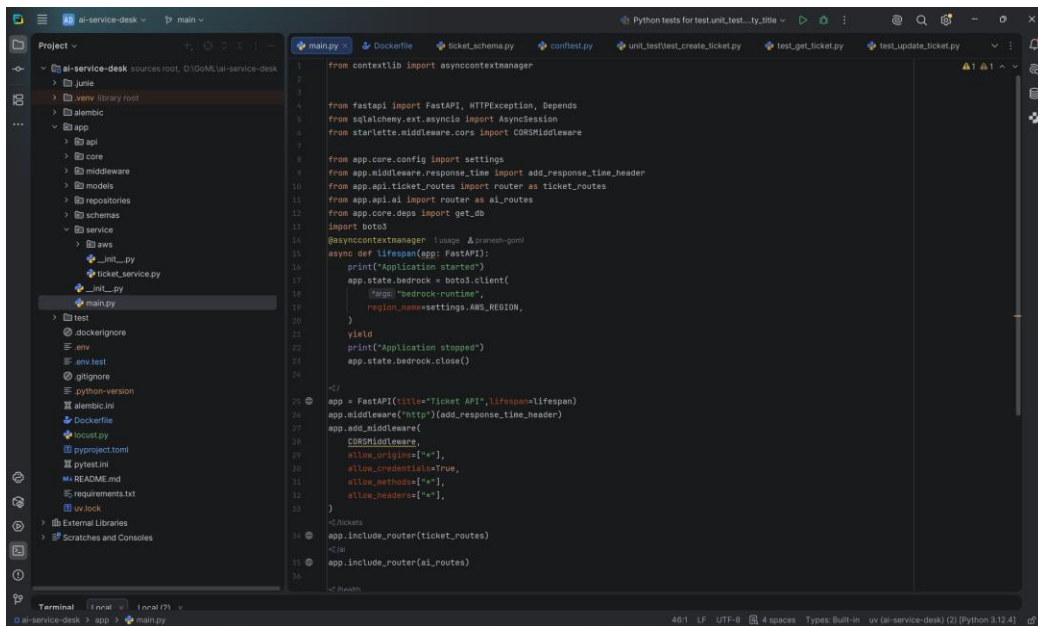
8.2 Configuration Files and Parameters

Configuration File / Parameter	Purpose
.env	Stores environment-specific configuration values.
DATABASE_URL	PostgreSQL database connection string.
AWS_ACCESS_KEY_ID	AWS access key used to access Amazon Bedrock.
AWS_SECRET_ACCESS_KEY	AWS secret access key for authentication.
AWS_REGION	Specifies the AWS region where Bedrock is available.

Configuration File / Parameter	Purpose
BEDROCK_MODEL_ID	Identifies the foundation model used for AI summary and response generation.
LOG_LEVEL	Configures the application's logging level.

9. Appendix

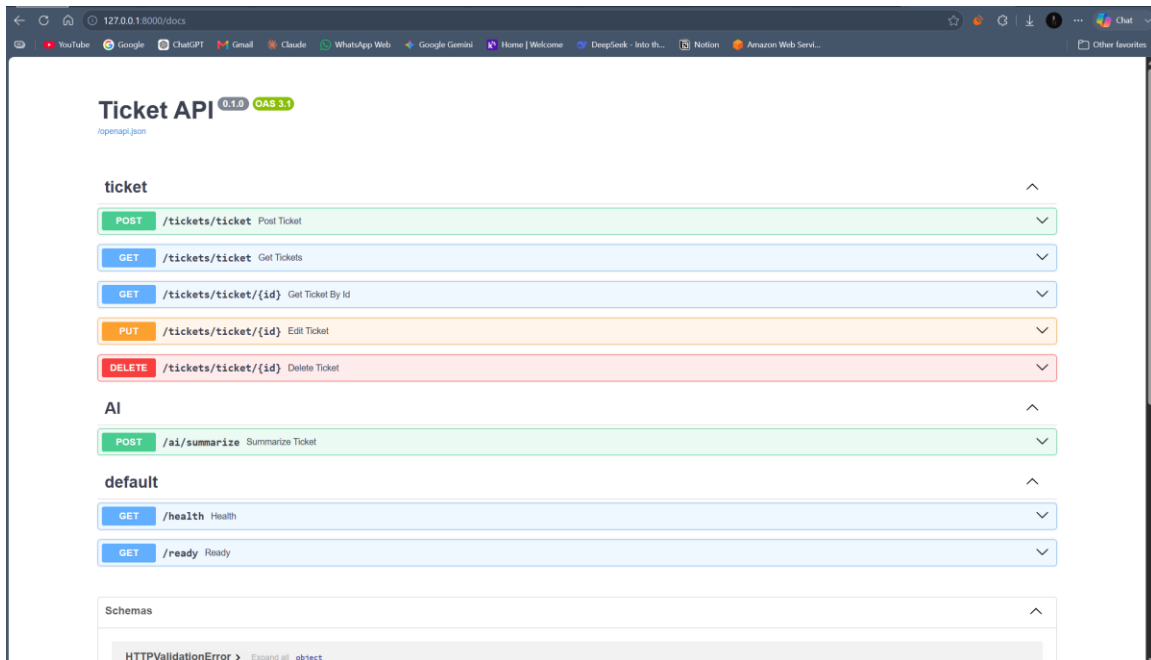
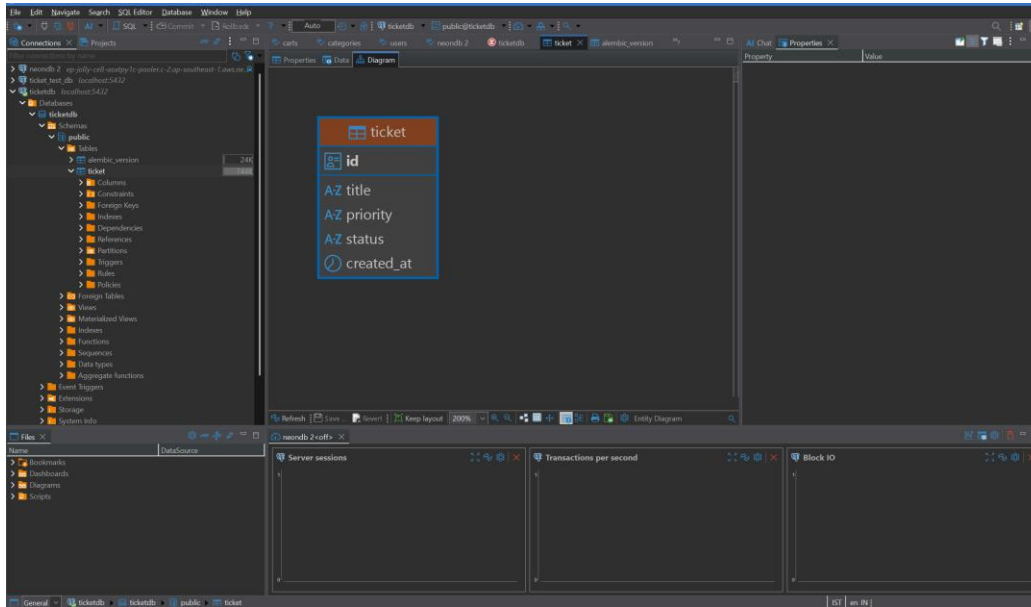
Screenshots



```

1 from contextlib import asynccontextmanager
2
3
4 from fastapi import FastAPI, HTTPException, Depends
5 from sqlalchemy.ext.asyncio import AsyncSession
6 from starlette.middleware.cors import CORSMiddleware
7
8 from app.core.config import settings
9 from app.middleware.response_time import add_response_time_header
10 from app.api.ticket_routes import router as ticket_routes
11 from app.api.ai import router as ai_routes
12 from app.core.deps import get_db
13
14 import boto3
15 @asynccontextmanager
16 async def lifespan(app: FastAPI):
17     print("Application starting")
18     app.state.bedrock = boto3.client(
19         "bedrock-runtime",
20         region_name=settings.AWS_REGION,
21     )
22     yield
23     print("Application stopping")
24     app.state.bedrock.close()
25
26
27 app = FastAPI(title="Ticket API", lifespan=lifespan)
28 app.middleware_stack = None
29 app.add_middleware(
30     CORSMiddleware,
31     allow_origins=["*"],
32     allow_credentials=True,
33     allow_methods=["*"],
34     allow_headers=["*"],
35 )
36
37
38 app.include_router(ticket_routes)
39
40
41 app.include_router(ai_routes)
42
43

```



Code Snippets

1. Ticket Route:

```
@router.post("/ticket", response_model=ticket_schema.TicketOutSchema, status_code=201)
async def post_ticket(ticket_in:
ticket_schema.TicketInSchema, db: AsyncSession = Depends(get_db)):
    service = TicketService(TicketRepository(db))
    try:
        ticket = await service.create_ticket(ticket_in)
        return {
            "message": "User Created successfully",
```

```
        "ticket": ticket
    }
    except DuplicateTicketError as exc:
        raise HTTPException(status_code=409, detail=str(exc))
```

2. Ticket model:

```
class Ticket(Base):
    __tablename__ = "ticket"
    id = Column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    title = Column(String, nullable=False)
    priority = Column(Enum(PriorityEnum, name="priority_enum"),
nullable=False)
    status = Column(Enum(StatusEnum, name="status_enum"),
nullable=False)
    description = Column(String, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, server_default=func.now(),
onupdate=func.now())
```

3. Ticket schema:

```
class TicketSchema(BaseModel):
    id: UUID
    title: str
    priority: str
    status: str
    description: str
    created_at: datetime
    updated_at: datetime
```

4. Ticket repo:

```
async def create_ticket(self, payload: TicketInSchema, description:
str):
    data = payload.model_dump()
    data["description"] = description
    ticket = Ticket(**data)
    self.db.add(ticket)
    await self.db.flush()
    await self.db.refresh(ticket)
    return ticket
```

5. Ticket Service:

```
async def create_ticket(self, ticket_input: TicketInSchema):
    existing = await self.repo.get_ticket_by_title(ticket_input.title)
    if existing is not None:
        raise DuplicateTicketError("Ticket with this title already exists")
    if ticket_input.status != StatusEnum.OPEN:
        raise HTTPException(status_code=400, detail="A Ticket can be raised only as a Open ticket.")
    bedrock = BedrockService()
    description = bedrock.generate_description(ticket_input)
    return await self.repo.create_ticket(ticket_input, description)
```

5. AI Route:

```
@router.post("/summarize", response_model=SummarizeResponse)
async def summarize_ticket(
    payload: SummarizeRequest,
    service: BedrockService = Depends(get_bedrock_service),
    db: AsyncSession = Depends(get_db)
) -> dict[str, str]:
    try:
        # Fetch the ticket list from repository
        try:
            ticket_repo = TicketRepository(db)
            tickets = await ticket_repo.get_all()

            # Format the tickets list nicely
            ticket_list_str = "\n".join([
                f"- Title: {t.title}, Priority: {t.priority.value if hasattr(t.priority, 'value') else t.priority}, Status: {t.status.value if hasattr(t.status, 'value') else t.status}, Description: {t.description}"
                for t in tickets
            ])
        except Exception as db_exc:
            print(f"Warning: Database query failed (PostgreSQL might be offline): {db_exc}")
            ticket_list_str = "(No existing tickets available. Database is offline/unreachable.)"

        return service.summarize_ticket(payload.ticket_description, ticket_list=ticket_list_str)
    except BedrockServiceError as exc:
        raise HTTPException(
            status_code=status.HTTP_502_BAD_GATEWAY,
            detail="The AI service is temporarily unavailable",
        ) from exc
```